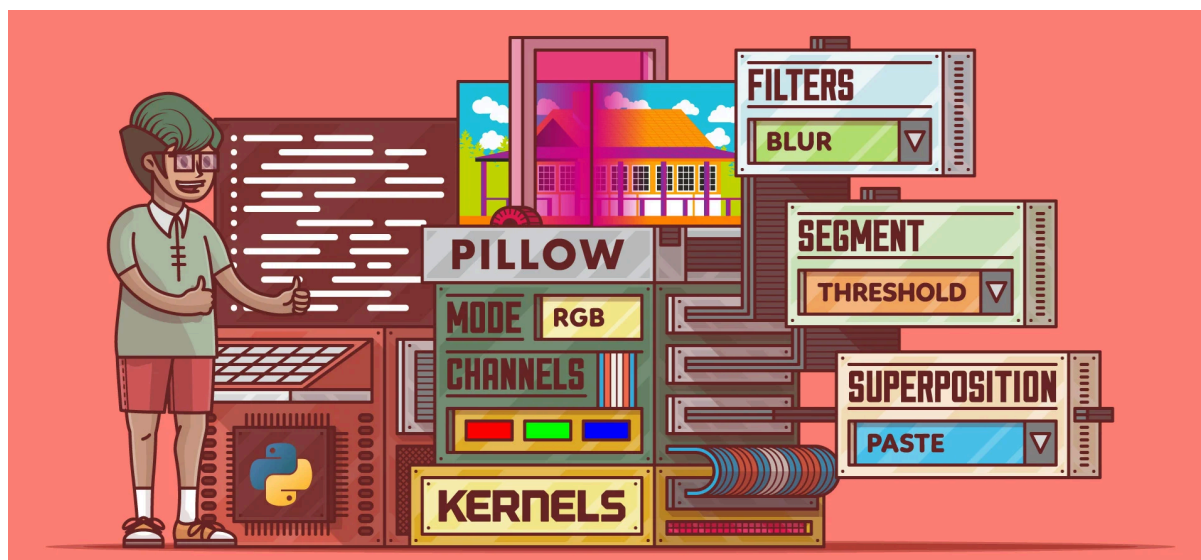


AirMouse Project



Indice

AirMouse Project.....	1
Introduzione.....	3
Gesture.....	3
Possibili miglioramenti.....	3
Problemi attuali.....	4
Applicazioni possibili.....	4
Librerie utilizzate.....	4
Schema degli eventi.....	5

Introduzione

Questo progetto implementa, con un sistema di visione artificiale, il controllo del cursore del mouse e il riconoscimento di gesture tramite il tracking delle mani per Linux. Utilizzando una webcam standard, il sistema rileva la posizione e i movimenti delle mani, traducendoli in comandi per il computer. L'applicazione è sviluppata in Python e si basa sulle librerie OpenCV e MediaPipe per l'elaborazione delle immagini e il riconoscimento delle gesture.

L'applicazione presenta un pannello di impostazioni nel quale si possono modificare in "live" alcuni parametri, inoltre è presente una funzione di reset ai valori iniziale e un tasto popup informativo su come funzionano le gesture.

È possibile inoltre, salvare i parametri modificati, i quali si caricheranno automaticamente all'avvio dell'applicazione, al fine di trovare i parametri giusti dopo alcuni test, per poter garantire un'esperienza utente ottimale.

Gesture

Le gesture disponibili sono le seguenti:

Click sinistro: per eseguirlo, si avvicina l'indice e il pollice della mano destra.

Trascinamento: per eseguirlo si tiene premuto l'indice e il pollice della mano destra per trascinare.

Click destro: per eseguirlo, si abbassa il mignolo della mano destra.

Zoom in: per eseguirlo, si allontanano gli indici delle mani destra e sinistra.

Zoom out: per eseguirlo, si avvicinano gli indici delle mani destra e sinistra.

Frecce direzionali: per eseguirlo, si chiude la mano sinistra a pugno e la si muove nella direzione desiderata per simulare le frecce direzionali, in particolare non è un vero proprio movimento, ma se il pugno viene rilevato in un posizione dello schermo, verrà attivata la freccia corrispondente alla posizione.

Possibili miglioramenti

I miglioramenti posso partire da una maggiore stabilità con filtri più avanzati per ridurre il tremolio del cursore e sfogliature ottimali per evitare falsi positivi

Seguirà il supporto per altri sistemi operativi come MacOS e Windows.

Problemi attuali

Latenza: Piccolo ritardo tra movimento fisico e risposta del cursore.

Illuminazione ambientale: Le performance sono ridotte in condizioni di scarsa illuminazione.

Falsi positivi: Occasionali attivazioni involontarie delle gesture.

Consumo risorse: Utilizzo CPU e RAM significativo durante l'elaborazione.

Applicazioni possibili

Controllo hands-free: Per utenti con disabilità motorie. Inoltre grazie alla possibilità di calibrare a piacimento varie impostazioni potrebbe essere un possibile approccio per persone con malattie come il parkinson, ponendo dei valori di sogliatura per bilanciare il tremolio della mano.

Presentazioni interattive: Controllare presentazioni (per esempio in PowerPoint) senza dispositivi fisici.

Ambienti sterili: Laboratori o sale operatorie dove il contatto fisico è limitato a causa dell'igiene.

Luoghi pubblici: Interfacce touchless per luoghi pubblici.

Librerie utilizzate

OpenCV: Elaborazione immagini in tempo reale dalla webcam (Licenza Apache (Alcune dipendenze di OpenCV potrebbero avere licenze diverse. Ad esempio, FFmpeg è rilasciato sotto la LGPLv2.1, mentre Qt5 è rilasciato sotto la LGPLv3. Queste licenze potrebbero imporre obblighi aggiuntivi, come la necessità di fornire il codice sorgente delle modifiche o di rendere pubblico il tuo codice se distribuito))

MediaPipe: Rilevazione e tracking delle mani con modelli pre addestrati (Licenza Apache)

PyAutoGUI: Controllo del cursore e simulazione input utente (Licenza BSD)

Tkinter: Interfaccia grafica per le impostazioni (Licenza BSD)

NumPy: Elaborazione numerica per filtri e trasformazioni (Licenza BSD)

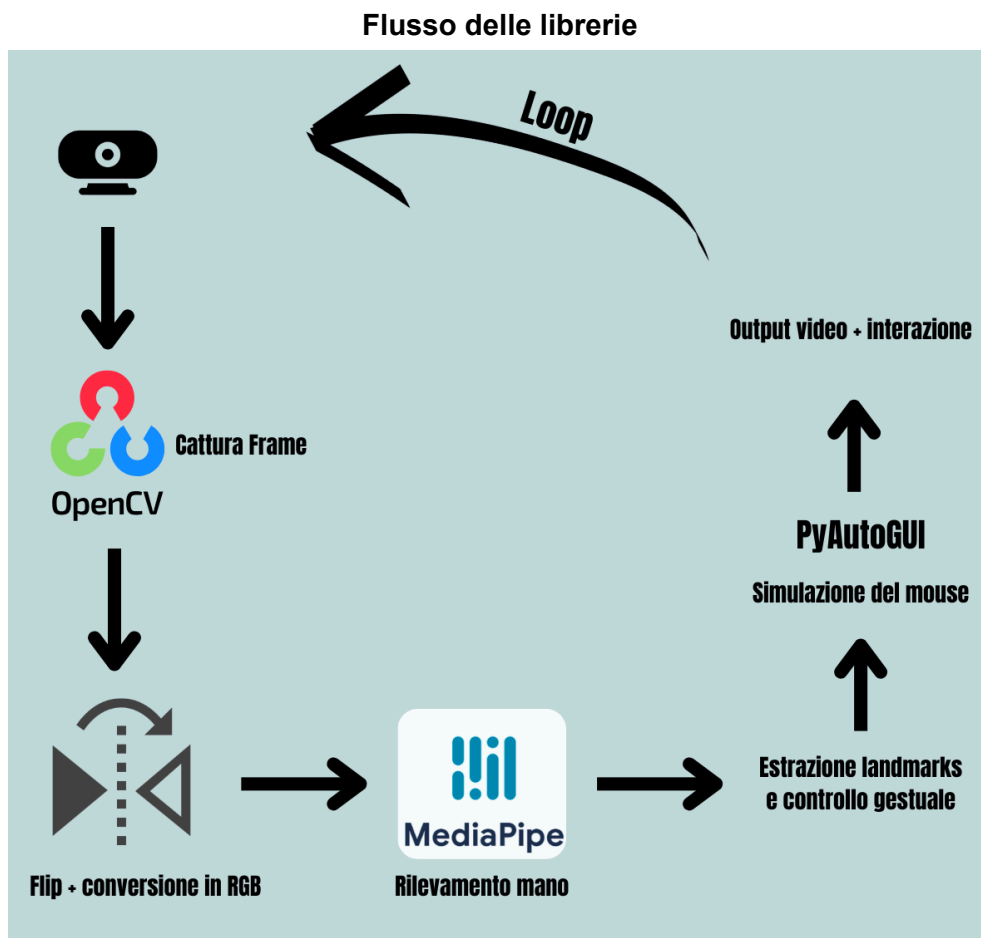
Schema degli eventi

Viene catturato frame per frame la riproduzione live della webcam, tramite OpenCV, vengono fatte alcune trasformazioni di preprocessing come il Flip e la conversione RGB, e l'output viene passato a MediaPipe, il quale estrae i landmarks.

I 21 landmarks corrispondono a punti anatomici specifici:

- 0:** Polso
- 1-4:** Pollice
- 5-8:** Indice
- 9-12:** Medio
- 13-16:** Anulare
- 17-20:** Mignolo

Dopodiché viene classificata la gesture in base ai landmarks riconosciuti, si esegue il comando e si ottiene il feedback visivo.



Flusso applicativo

